Modeling and Control of a Double Pendulum

Konstantinos Spinos

***Abstract*—Trajectory optimization (TO) is a technique or a sub-process of a general purpose - motion planning. In practice, motion planning comes with many problems including computational complexity, uncertainty, and physical constraints. To address these issues, a common approach is to use Trajectory Optimization methods. TO focuses on finding the best path that minimizes a given cost function (most likely time or energy) while respecting the physical constraints of the system. The optimized trajectory will later be used to control the system via Time Variant Linear Quadratic Regulator (TV-LQR) - an LQR controller designed for discrete time purposes.**

I. Introduction

The paper concerns the modeling, trajectory optimization, and control of a double pendulum system, as seen in Figure 1. More precisely, the trajectory optimization is to be solved with the Hermite-Simpson method, while the system will be controlled via TV-LQR.

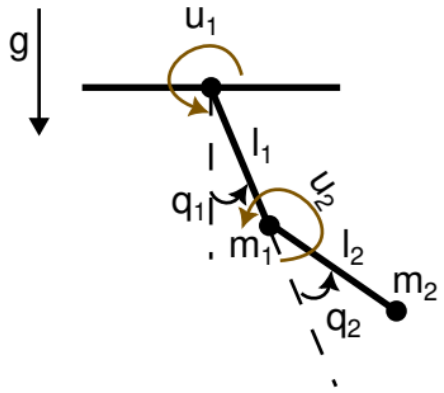


Fig. 1. Double Pendulum System

II. Modeling

The system consists of two links/rods of mass m_1, m_2 and length l_1, l_2 respectively. Both joints exert torque (u_1 and u_2 respectively). The system has four continuous state variables: two joint positions (q_1, q_2) and two joint velocities ($\dot{q}_1, \dot{q}_2$). The dynamics of the system along with the state and the control input is as follows:

$$\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u}) = \begin{bmatrix} \dot{q}_1 \\ \dot{q}_2 \\ \ddot{q}_1 \\ \ddot{q}_2 \end{bmatrix} \in \mathbb{R}^4$$

$$\mathbf{x} = \begin{bmatrix} q_1 \\ q_2 \\ \dot{q}_1 \\ \dot{q}_2 \end{bmatrix} \in \mathbb{R}^4 \quad \mathbf{u} = \begin{bmatrix} u_1 \\ u_2 \end{bmatrix} \in \mathbb{R}^2$$

By defining $\mathbf{q} = \begin{bmatrix} q_1 \\ q_2 \end{bmatrix}$ we can derive the following forward dynamics equation:

$$\ddot{\mathbf{q}} = \mathbf{M}^{-1}(\mathbf{u} - \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{G}(\mathbf{q}))$$

where

$$\mathbf{M} = \begin{bmatrix} l_1^2 m_1 + l_2^2 m_2 + l_1^2 m_2 + 2 l_1 m_2 l_2 \cos q_2 & l_2^2 m_2 + l_1 m_2 l_2 \cos q_2 \\ l_2^2 m_2 + l_1 m_2 l_2 \cos q_2 & l_2^2 m_2 \end{bmatrix}$$

$$\mathbf{C} = \begin{bmatrix} -2\dot{q}_2 l_1 m_2 l_2 \sin q_2 & -\dot{q}_2 l_1 m_2 l_2 \sin q_2 \\ \dot{q}_1 l_1 m_2 l_2 \sin q_2 & 0 \end{bmatrix}$$

$$\mathbf{G} = \begin{bmatrix} -g m_1 l_1 \sin q_1 - g m_2 (l_1 \sin q_1 + l_2 \sin(q_1 + q_2)) \\ -g m_2 l_2 \sin(q_1 + q_2) \end{bmatrix}$$

and $g = 9.81\, m/s^2$, $m_1 = m_2 = 1\, kg$, $l_1 = l_2 = 0.5\, m$. The dimension of each matrix is: $\mathbf{M} \in \mathbb{R}^{2\times2}, \mathbf{C} \in \mathbb{R}^{2\times2}, \mathbf{G} \in \mathbb{R}^{2\times1}$.

A simple visualization of the system can be seen below (initial state $x_{init} = [0.3 \quad 0.9 \quad 0 \quad 0]^T$). The system has been discretized by the Runge-Kutta 4th Order integrator.

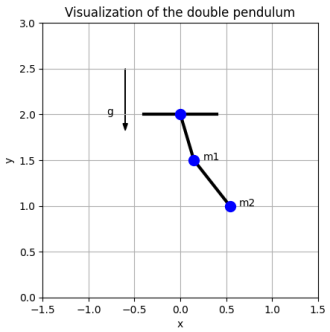
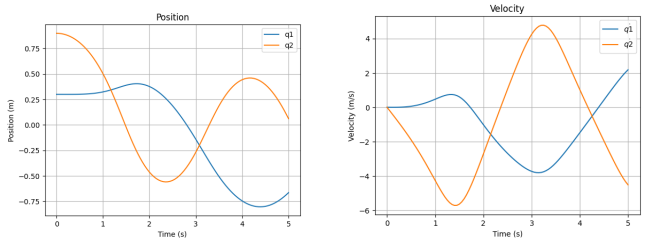


Fig. 2. System Visualization



(a) Position (b) Velocity

Fig. 3. Angle's q_1 and q_2 position and velocity tracking

III. Trajectory Optimization

In this section, our aim is to build an optimized trajectory for the pendulum system. It's initial and final state will respectfully be:

$$\mathbf{x}_0 = \begin{bmatrix} 0 & 0 & 0 & 0 \end{bmatrix}^T \qquad \mathbf{x}_f = \begin{bmatrix} \pi & 0 & 0 & 0 \end{bmatrix}^T$$

The double pendulum will thus begin at the bottom vertical, where the only force applied is gravity, while the goal is to enforce a swing-up behavior and move the pendulum system at the top vertical point. Physical limitations of the pendulums require the control inputs to be in bounds: $-10 \leq \mathbf{u} \leq 10$. In addition to the physical constraints of the system, we need to take into account the boundary conditions as well as the constraints applied by the Hermite-Simpson method. This method imports the dynamic constraints above:

$$\mathbf{x}_{k+1} - \mathbf{x}_k = \frac{1}{6} h (f_k + 4 f_{k+\frac{1}{2}} + f_{k+1})$$
$$\mathbf{x}_{k+\frac{1}{2}} = \frac{1}{2}(\mathbf{x}_{k+1} + \mathbf{x}_k) + \frac{h}{8}(f_k - f_{k+1})$$

We have two options:

1) Optimize over the variables $\mathbf{x}, \mathbf{u}$ solely, and merge the constraints into one
2) Add mid points $\mathbf{x}_{k+\frac{1}{2}}, \mathbf{u}_{k+\frac{1}{2}}$ as optimization variables and confront separately the two constraints

The first option provides faster evaluation of the cyipopt problem but is more unstable and vulnerable to noise. We choose the second option for a more robust approach. As a result, the optimization variable $\mathbf{z}$ is as seen below.

$$\mathbf{z} = \begin{bmatrix} \mathbf{x} \\ \mathbf{u} \\ \mathbf{x}_m \\ \mathbf{u}_m \end{bmatrix} = \begin{bmatrix} x_0 \\ x_1 \\ \vdots \\ x_K \\ u_0 \\ u_1 \\ \vdots \\ u_K \\ x_{\frac{1}{2}} \\ \vdots \\ x_{K-\frac{1}{2}} \\ u_{\frac{1}{2}} \\ \vdots \\ u_{K-\frac{1}{2}} \end{bmatrix} \in \mathbb{R}^{1206}$$

$\mathbf{x} \in \mathbb{R}^{K \times N}, \mathbf{u} \in \mathbb{R}^{K \times M}, \mathbf{x}_m \in \mathbb{R}^{(K-1) \times N}, \mathbf{u}_m \in \mathbb{R}^{(K-1) \times M}$ and $N = 6$ (state size), $M = 2$ (control size), $K = 101$ (knot points).

After defining the optimization variable and the constraints of the problem, we shall choose an appropriate cost function. Since we desire to build a trajectory path for the double pendulum system, the best overall approach is to use a minimum effort controller, of the form:

$$\mathbf{J} = \sum_{k=0}^{K} \|\mathbf{u}\|^2 dt, \quad dt = 0.05 sec$$

We could include a weight matrix $\mathbf{R}$, however for just trajectory planning only (no control), it is unnecessary.
What remains is to transfer this analysis to the cyipopt problem with the structure below.

```
class TrajOpt:
    def objective():
        return ...

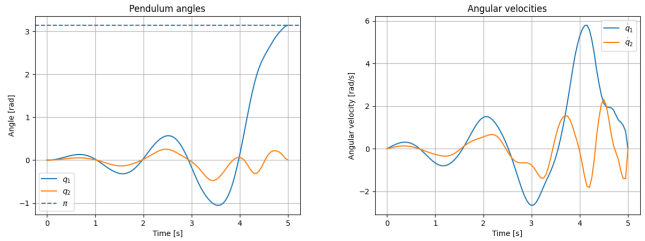
    def gradient():
        return ...

    def constraints():
        return ...

    def jacobian():
        return ...

    def jacobianstructure(self):
        return ...
```

The *objective* function returns the cost function to minimize, the *gradient* function returns the gradient of the cost, the *constraints* function returns all of the problem's constraints, the *jacobian* function returns only the non-zeros of the jacobian matrix of the constraints (mandatory for so many optimization variables - it reduces runtime and offers faster convergence) and the *jacobianstructure* function is there to ensure that we take the sparse jacobian. The results can be seen in the plot figures below.



(a) Position (b) Velocity

Fig. 4. Angles q_1, q_2 and velocities $\dot{q}_1, \dot{q}_2$

The algorithm converged to the desired state after 45 iterations, reaching 9.523e-07 in the infeasibility factor and 1.1653e-11 in the objective, which implies that in the end, the forces applied to the pendulum system were almost zero.

```
...
Iter #43 | Obj: 1.419e-06 | Infeasibility: 9.9688e-07
Iter #44 | Obj: 1.5971e-09 | Infeasibility: 7.3914e-07
Iter #45 | Obj: 1.1653e-11 | Infeasibility: 9.523e-07
SUCCESS: Optimal Solution Found!
```

Fig. 5. Cyipopt converged

IV. Trajectory Optimization

In this section we focus to control the pendulum system via Time-Variant LQR (TVLQR) using the optimal trajectory computed earlier. TVLQR is designed to track moving trajectories that require linearization at every timestep of the simulation. The first step is to linearize dynamics around the optimal states and controls, deriving the matrices above.

$$\boldsymbol{A}_k = \frac{\partial f_\mathrm{d}}{\partial \boldsymbol{x}}\bigg|_{\bar{\boldsymbol{x}},\,\bar{\boldsymbol{u}}} \qquad \boldsymbol{B}_k = \frac{\partial f_\mathrm{d}}{\partial \boldsymbol{u}}\bigg|_{\bar{\boldsymbol{x}},\,\bar{\boldsymbol{u}}}$$

where f_d stands for the discretized dynamics function of the system through Runke-Kutta 4th Order. The optimal states and controls $\bar{\mathbf{x}}, \bar{\mathbf{u}}$ are computed from the trajectory. By defining proper weight matrices $\mathbf{Q}, \mathbf{R}$ and $\mathbf{Q}_N$, we then solve the two equations from the TVLQR algorithm as seen below.

$$\mathbf{K}_k = (\mathbf{R}_k + \mathbf{B}_k^T \mathbf{P}_{k+1} \mathbf{B}_k)^{-1} \mathbf{B}_k^T \mathbf{P}_{k+1} \mathbf{A}_k$$
$$\mathbf{P}_k = \mathbf{Q}_k + \mathbf{A}_k^T \mathbf{P}_{k+1} (\mathbf{A}_k - \mathbf{B}_k \mathbf{K}_k)$$

We extract only the gains $\mathbf{K}_k$, needed for the controller later on. This algorithm solves the regular LQR control with the only difference being that the goal is to follow a moving path, not a static state. After formulating the core path of this section, the feedback controller remains to be defined. For a more realistic analysis, Gaussian noise will be added to the system, meaning that the controller will be of the form:

$$\mathbf{u} = \bar{\mathbf{u}}_k - \mathbf{K}_k(\mathbf{x}_{measured} - \bar{\mathbf{x}}_k)$$

where $\bar{\mathbf{u}}_k, \bar{\mathbf{x}}_k$ are the nominal controls and states, and $\mathbf{x}_{measured}$ consists of the current sate and the noisy one. This feedback law ensures the pendulum to follow the trajectory under the influence of noise. Of course, how well this theory is verified strongly depends on the level of noise. Below is displayed an example of two cases where the controller tries to follow the trajectory path under Gaussian noise with zero mean and normal distributions $\sigma_1 = 0.001$ and $\sigma_2 = 0.5$. The difference between low noise and high noise is visible.

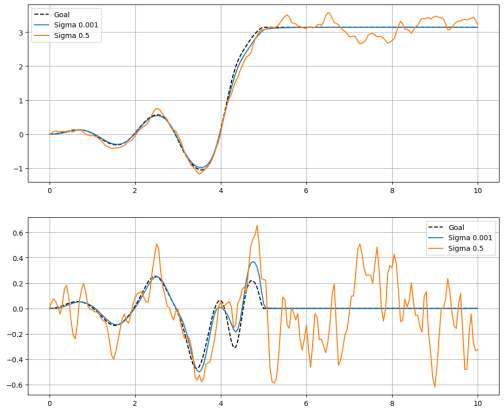


Fig. 6. $\sigma_1 = 0.001$ and $\sigma_2 = 0.5$

We notice that for a normal distribution value $\sigma = 0.001$, the noise is almost unnoticeable. In the opposite, increasing the value to $\sigma = 0.5$ affects dramatically the correct tracking of the trajectory. It should be noted that the trajectory runs for a total time of 5 seconds. After 5 seconds, the controller "shuts down" since it has nothing to follow. To this end, we add an Infinite horizon LQR controller to try and stabilize the double pendulum system in the top equilibrium point. Infinite horizon LQR is generally better in terms of stabilization compared to the standard form of LQR control, mostly because $\mathbf{K}_\infty$ is calculated to minimize cost over an infinite time window. In the above, it can be witnessed the case where noise is set to zero.

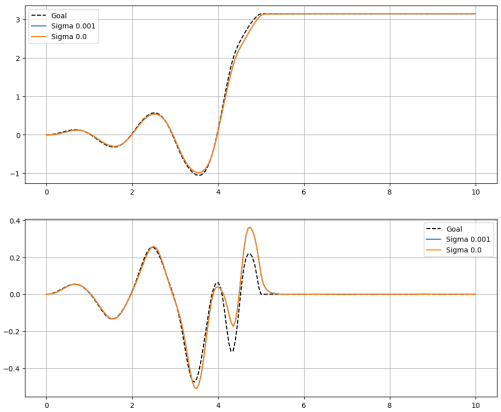


Fig. 7. $\sigma_1 = 0.001$ and $\sigma_2 = 0$

We notice that the controller cannot track the trajectory with perfect precision, for various reasons. The double pendulum system is a highly nonlinear system that we linearize using Taylor Series. In addition, we ignore the high-order terms from the Taylor expansion. It is natural for the controller path to diverge when the pendulum moves fast or swings wide.
A last example for visual purposes is seen below, for $\sigma_1 = 0.05, \sigma_2 = 0.2$.

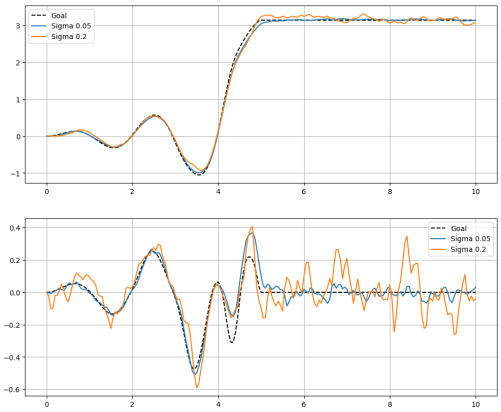


Fig. 8. $\sigma_1 = 0.05$ and $\sigma_2 = 0.2$